

Министерство науки и высшего образования РФ
Федеральное государственное автономное образовательное учреждение
высшего образования «Национальный исследовательский технологический
университет «МИСиС»
Институт ИТКН
Кафедра АСУ

**ОТЧЕТ ПО УЧЕБНОЙ/ ПРОИЗВОДСТВЕННОЙ
ПРАКТИКЕ**

Тема: разработка веб-сервиса управления задачами
на основе Kanban-досок

Выполнил:
ст. гр. БИВТ-24-9
Кукишев Артём Вадимович

Москва 2026

Содержание

Введение

1 Постановка задачи

1.1 Функциональные требования

1.2 Нефункциональные требования

1.3 Выбор технологий

2 Проектирование системы

2.1 Архитектура решения

2.2 Модель данных

2.3 Проектирование программного интерфейса

3 Реализация системы

3.1 Структура проекта

3.2 Модель авторизации

3.3 Ролевая модель

3.4 Транзакции и защита от коллизий

3.5 Журнал изменений

3.6 Клиентское приложение

3.7 Развёртывание

4 Тестирование

Заключение

Список использованных источников

Введение

В современной разработке программного обеспечения задачи организации командной работы всё чаще решаются с помощью систем визуального планирования. Одним из наиболее распространённых подходов является методология Kanban, при которой работа представлена в виде карточек, перемещающихся по колонкам, отражающим стадии выполнения. Такой способ представления делает состояние процесса наглядным: в любой момент видно, какие задачи находятся в очереди, какие выполняются и какие завершены.

Системы, реализующие эту методологию, такие как Trello, Kaiten и Jira, широко применяются в командах разработки. Однако использование внешних сервисов не всегда возможно: организация может предъявлять требования к хранению данных внутри собственного контура, к интеграции с существующими системами или к специфической функциональности. Это обуславливает актуальность разработки собственных решений подобного класса.

Целью данной практики являлась разработка многопользовательского веб-сервиса Kanban-досок, функционально аналогичного существующим коммерческим решениям. Для достижения поставленной цели решались следующие задачи: анализ требований и проектирование модели данных, разработка серверной части с программным интерфейсом и механизмом разграничения доступа, реализация клиентского приложения с поддержкой перетаскивания карточек, обеспечение целостности данных при одновременной работе нескольких пользователей, а также подготовка воспроизводимого развёртывания системы.

Практическая значимость работы состоит в том, что полученное решение представляет собой законченный программный продукт, который может быть развёрнут в инфраструктуре организации и использован для управления задачами внутри команды.

1 Постановка задачи

1.1 Функциональные требования

В рамках практики необходимо реализовать веб-сервис управления задачами на основе Kanban-досок. Требования к обязательному минимуму реализации сформулированы следующим образом:

- 1) Пользователи и доступ: регистрация и вход в систему; доска имеет владельца и участников.
- 2) Доски: создание, редактирование, удаление; получение списка досок пользователя.
- 3) Колонки: создание, редактирование, удаление.
- 4) Карточки: создание и редактирование заголовка и описания; перемещение между колонками методом перетаскивания; сохранение порядка карточек в колонке; назначение исполнителя; установка срока выполнения.
- 5) Комментарии: добавление комментариев к карточкам.
- 6) Роли: не менее трёх ролей с разграничением прав по принципу минимальных привилегий.

В качестве возможных расширений решения рассматривались: метки карточек, чек-листы, вложения, журнал изменений, поиск по карточкам и шаблоны досок.

1.2 Нефункциональные требования

- Пароли пользователей должны храниться в базе данных исключительно в виде хэшей.
- Обмен учётных данных на токен доступа; все прочие операции выполняются с использованием токена.
- Операции, затрагивающие несколько записей, должны выполняться в транзакциях.
- Система должна быть защищена от коллизий при одновременном редактировании доски несколькими клиентами.

- Обращения к базе данных выполняются только через серверную часть.
- Развёртывание системы должно быть воспроизводимым.

1.3 Выбор технологий

Задание не накладывает ограничений на выбор технологий, поэтому решение принималось исходя из характера задачи. Обоснование выбора приведено в Таблице 1.

Таблица 1 — Обоснование выбора технологий

Компонент	Выбор	Обоснование
Язык серверной части	Python 3.12	Развитая экосистема, высокая скорость разработки
Веб-фреймворк	FastAPI	Автоматическая валидация данных и генерация документации OpenAPI
СУБД	PostgreSQL 16	Полная поддержка транзакций, тип JSONB, надёжность
Работа с БД	SQLAlchemy 2.0	Декларативное описание моделей, независимость от диалекта
Миграции	Alembic	Версионирование схемы базы данных
Хэширование	bcrypt	Адаптивная функция со встроенной солью
Клиентская часть	React 18	Компонентный подход, широкая распространённость
Перетаскивание	@dnd-kit	Современная библиотека с поддержкой доступности
Развёртывание	Docker Compose	Воспроизводимость окружения

2 Проектирование системы

2.1 Архитектура решения

Сервис построен по классической трёхзвенной архитектуре, включающей клиентское приложение, сервер приложений и базу данных. Клиентское приложение реализовано в виде одностраничного приложения на библиотеке React. Серверная часть построена на фреймворке FastAPI и предоставляет программный интерфейс в стиле REST. В качестве системы управления базами данных используется PostgreSQL. Общая схема взаимодействия компонентов приведена на Рис. 1.

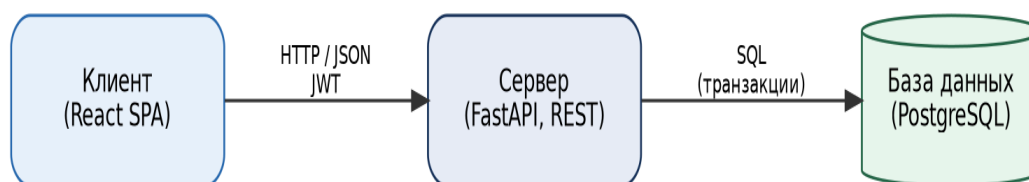


Рис. 1. Верхнеуровневая архитектура сервиса

Ключевой принцип архитектуры состоит в том, что клиентское приложение не обращается к базе данных напрямую — все операции проходят через серверную часть. Это обеспечивает единую точку контроля доступа, валидации входных данных и управления транзакциями. Взаимодействие клиента и сервера осуществляется по протоколу HTTP в формате JSON, аутентификация — с помощью токенов JWT.

Серверная часть разделена на слои: слой моделей описывает структуру данных, слой схем отвечает за валидацию входных и выходных данных, слой сервисов содержит бизнес-логику, слой маршрутов реализует конечные точки программного интерфейса. Такое разделение упрощает сопровождение и тестирование системы.

2.2 Модель данных

Модель данных включает семь сущностей: пользователи, доски, участники досок, колонки, карточки, комментарии и журнал изменений. Связь «пользователь — доска» имеет тип «многие-ко-многим» и разрешена через промежуточную таблицу участников, в которой дополнительно хранится роль пользователя на конкретной доске. Отдельная таблица журнала изменений фиксирует историю действий над доской. Диаграмма «сущность — связь» представлена на Рис. 2.

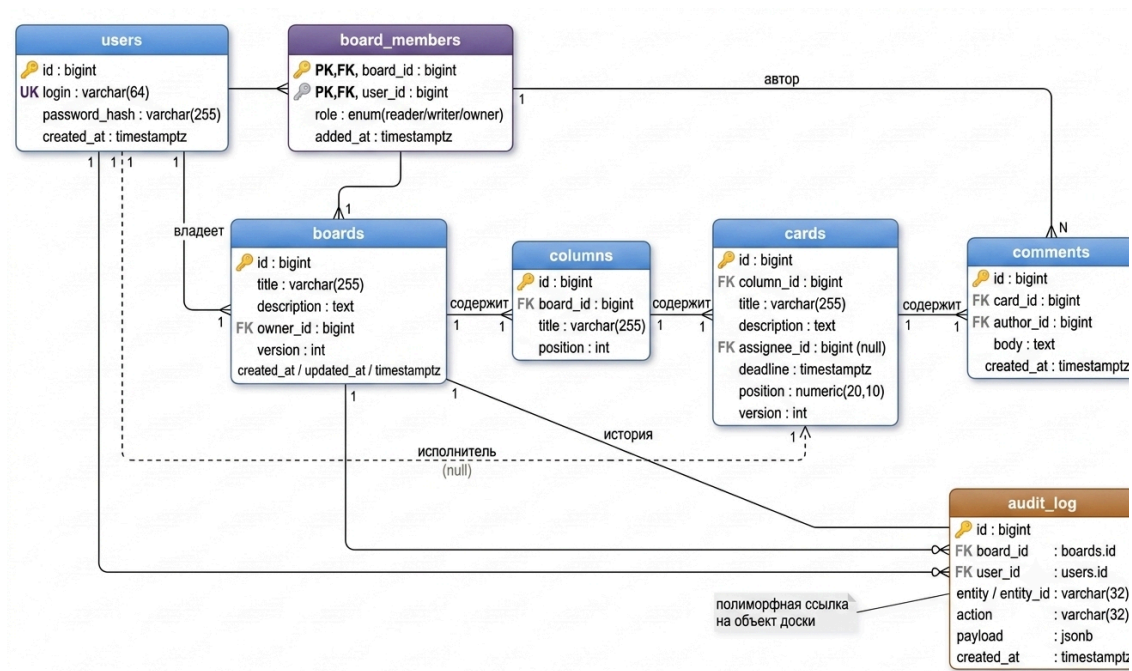


Рис. 2. ER-диаграмма базы данных

Перечень связей между сущностями и их кардинальности приведены в Таблице 2.

Таблица 2 — Связи между сущностями

Связь	Тип	Реализация
Пользователь — доска (владение)	1:N	boards.owner_id
Пользователь — доска (участие)	N:N	board_members, поле role
Доска — колонка	1:N	columns.board_id, каскадное удаление
Колонка — карточка	1:N	cards.column_id, каскадное удаление
Пользователь — карточка (исполнитель)	1:N	cards.assignee_id, обнуление
Карточка — комментарий	1:N	comments.card_id, каскадное удаление
Доска — запись журнала	1:N	audit_log.board_id

Первичные ключи всех таблиц — суррогатные, целочисленные, с автоинкрементом. Внешние ключи снабжены правилами ссылочной целостности: при удалении доски удаляются её колонки, карточки и комментарии; при удалении пользователя ссылка на него как на исполнителя обнуляется, что позволяет сохранить саму карточку.

Индексы

Для ускорения частых выборок созданы индексы, перечисленные в Таблице 3.

Таблица 3 — Индексы базы данных

Индекс	Поля	Назначение
idx_board_members_user	user_id	Список досок пользователя
idx_columns_board	board_id, position	Отрисовка колонок в порядке
idx_cards_column	column_id, position	Отрисовка карточек в порядке
idx_cards_assignee	assignee_id	Фильтрация по исполнителю
idx_cards_deadline	deadline	Поиск просроченных задач
idx_comments_card	card_id, created_at	Лента комментариев
idx_audit_board_time	board_id, created_at	Просмотр журнала изменений

Фрагмент схемы базы данных

Ниже приведён фрагмент описания схемы на языке SQL для ключевых таблиц.

```
CREATE TABLE boards (  
    id          BIGSERIAL PRIMARY KEY,  
    title       VARCHAR(255) NOT NULL,  
    description TEXT,  
    owner_id    BIGINT NOT NULL REFERENCES users(id)  
                ON DELETE CASCADE,  
    version     INT NOT NULL DEFAULT 1,  
    created_at  TIMESTAMPTZ NOT NULL DEFAULT now(),  
    updated_at  TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TYPE board_role AS ENUM ('reader', 'writer', 'owner');  
  
CREATE TABLE board_members (  
    board_id BIGINT REFERENCES boards(id) ON DELETE CASCADE,  
    user_id  BIGINT REFERENCES users(id)  ON DELETE CASCADE,  
    role     board_role NOT NULL DEFAULT 'reader',  
    PRIMARY KEY (board_id, user_id)  
);  
  
CREATE TABLE cards (  
    id          BIGSERIAL PRIMARY KEY,  
    column_id   BIGINT NOT NULL REFERENCES columns(id)  
                ON DELETE CASCADE,  
    title       VARCHAR(255) NOT NULL,  
    description TEXT,  
    assignee_id BIGINT REFERENCES users(id) ON DELETE SET NULL,  
    deadline    TIMESTAMPTZ,  
    position    NUMERIC(20,10) NOT NULL,  
    version     INT NOT NULL DEFAULT 1  
);
```

Листинг 1. Фрагмент схемы базы данных

2.3 Проектирование программного интерфейса

Программный интерфейс реализован в стиле REST. Ресурсы адресуются иерархически: колонки и карточки вложены в доску, комментарии — в карточку. Все конечные точки, за исключением регистрации и входа, требуют передачи токена в заголовке запроса. Перечень основных методов приведён в Таблице 4.

Таблица 4 — Основные методы программного интерфейса

Метод	Конечная точка	Назначение	Мин. роль
POST	/api/auth/register	Регистрация	—
POST	/api/auth/login	Вход, выдача токена	—
GET / POST	/api/boards	Список и создание досок	—
GET / PATCH / DELETE	/api/boards/{id}	Операции с доской	reader / writer / owner
GET / POST	/api/boards/{id}/members	Участники доски	reader / owner
PATCH / DELETE	/api/boards/{id}/members/{uid}	Роль участника	owner
GET / POST	/api/boards/{id}/columns	Колонки	reader / writer
PATCH / DELETE	/api/columns/{id}	Изменение колонки	writer
GET / POST	/api/boards/{id}/cards	Карточки	reader / writer
PATCH / DELETE	/api/cards/{id}	Изменение карточки	writer
POST	/api/cards/{id}/move	Перемещение карточки	writer
GET / POST	/api/cards/{id}/comments	Комментарии	reader / writer
GET	/api/boards/{id}/audit	Журнал изменений	owner

Документация программного интерфейса формируется автоматически на основе описанных схем данных и доступна в интерактивном виде по адресу /docs.

3 Реализация системы

3.1 Структура проекта

Исходный код организован в виде двух подпроектов — серверного и клиентского. Структура каталогов приведена в Листинге 2.

kanban/

<code>docker-compose.yml</code>	конфигурация развёртывания
<code>backend/</code>	
<code>app/</code>	
<code>models/</code>	модели данных (SQLAlchemy)
<code>schemas/</code>	схемы валидации (Pydantic)
<code>core/</code>	безопасность, проверка прав
<code>services/</code>	бизнес-логика, журнал изменений
<code>api/</code>	маршруты REST
<code>main.py</code>	точка входа приложения
<code>migrations/</code>	миграции Alembic
<code>scripts/seed.py</code>	наполнение базы данными
<code>frontend/</code>	
<code>src/</code>	
<code>pages/</code>	страницы приложения
<code>components/</code>	компоненты интерфейса
<code>context/</code>	хранение состояния авторизации
<code>api/client.js</code>	обёртка над HTTP-запросами
<code>docs/</code>	отчётные материалы

Листинг 2. Структура проекта

3.2 Модель авторизации

Аутентификация пользователей основана на токенах JWT. При входе пользователь передаёт логин и пароль, а в ответ получает подписанный токен, который затем прикладывается к каждому защищённому запросу. Сервер проверяет подпись и срок действия токена, извлекает идентификатор пользователя и определяет его права. Жизненный цикл токена показан на Рис. 3.

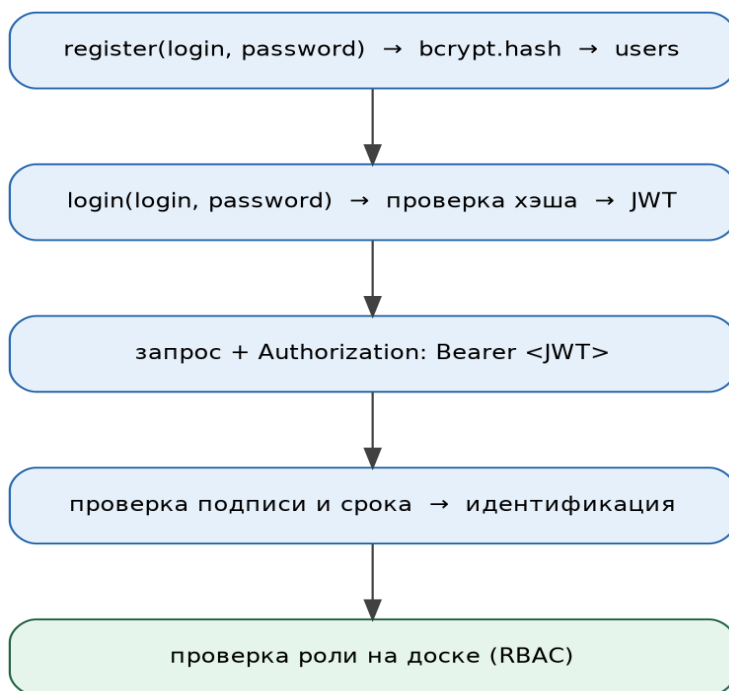


Рис. 3. Жизненный цикл токена авторизации

Пароли пользователей никогда не хранятся в открытом виде. При регистрации пароль хэшируется алгоритмом bcrypt со встроенной солью, и в базе данных сохраняется только полученный хэш. При входе введенный пароль повторно хэшируется и сравнивается с сохранённым значением. Такой подход исключает компрометацию паролей даже при получении злоумышленником доступа к базе данных. Реализация функций работы с паролями и токенами приведена в Листинге 3.

```
def hash_password(password: str) -> str:
    return bcrypt.hashpw(password.encode(),
                           bcrypt.gensalt()).decode()

def verify_password(password: str, password_hash: str) -> bool:
    return bcrypt.checkpw(password.encode(),
                           password_hash.encode())

def create_token(user_id: int) -> str:
    payload = {
        'sub': str(user_id),
        'exp': datetime.now(timezone.utc) + timedelta(hours=24),
        'iat': datetime.now(timezone.utc),
    }
    return jwt.encode(payload, settings.jwt_secret,
                      algorithm='HS256')
```

Листинг 3. Работа с паролями и токенами

Коды ответов сервера при различных ситуациях, связанных с доступом, приведены в Таблице 5.

Таблица 5 — Коды ответов при ошибках доступа

Ситуация	Код ответа
Токен отсутствует или некорректен	401 Unauthorized
Срок действия токена истёк	401 Unauthorized
Недостаточно прав для действия	403 Forbidden
Нет доступа к доске	404 Not Found
Конфликт версий при редактировании	409 Conflict

3.3 Ролевая модель

Для разграничения прав доступа выбрана модель управления доступом на основе ролей (RBAC). Пользователю на уровне доски назначается одна из трёх ролей, образующих иерархию по объёму прав: читатель, писатель, владелец. Права ролей приведены в Таблице 6.

Таблица 6 — Роли и права доступа

Роль	Чтение	Редактирование	Администрирование	Управление ролями
Читатель (reader)	Да	Нет	Нет	Нет
Писатель (writer)	Да	Да	Нет	Нет
Владелец (owner)	Да	Да	Да	Да

Реализован принцип минимальных привилегий: доска, к которой у пользователя нет доступа, возвращает ответ «не найдено» вместо «доступ запрещён», что не раскрывает сам факт её существования. Проверка прав вынесена в отдельную функцию, используемую всеми защищёнными маршрутами; её реализация приведена в Листинге 4.

```
ROLE_ORDER = {'reader': 1, 'writer': 2, 'owner': 3}

def require_board(db, board_id, user, min_role):
    board = db.get(Board, board_id)
    if board is None:
        raise HTTPException(404, 'Доска не найдена')
    role = get_role(db, board_id, user.id)
    if role is None:
        # чужая доска неотличима от несуществующей
        raise HTTPException(404, 'Доска не найдена')
    if ROLE_ORDER[role] < ROLE_ORDER[min_role]:
        raise HTTPException(403, 'Недостаточно прав')
    return board
```

Листинг 4. Проверка прав доступа к доске

Выбор в пользу RBAC, а не модели на основе атрибутов (ABAC), обусловлен характером предметной области. Модель ABAC накладывает доступ на комбинацию атрибутов пользователя и ресурса и оправдана там, где правила зависят от множества динамических факторов: времени, подразделения, уровня конфиденциальности. В рассматриваемой задаче полный набор прав описывается тремя фиксированными ролями, привязанными к паре «пользователь — доска», а динамических атрибутивных условий не возникает. Таким образом, RBAC обеспечивает необходимую выразительность при существенно меньшей сложности реализации и сопровождения.

3.4 Транзакции и защита от коллизий

Операции, затрагивающие несколько записей базы данных, выполняются в рамках одной транзакции. Характерным примером является перемещение карточки: изменение колонки, обновление позиции, увеличение версии и запись в журнал изменений фиксируются атомарно, то есть либо применяются целиком, либо не применяются вовсе.

Для защиты от конфликтов при одновременном редактировании применяется механизм оптимистичной блокировки на основе поля версии. Клиент передаёт известную ему версию объекта; если она не совпадает с текущей версией в базе данных, сервер возвращает ответ с кодом 409, после чего клиент перечитывает актуальное состояние и повторяет действие. Последовательность взаимодействия показана на Рис. 4.

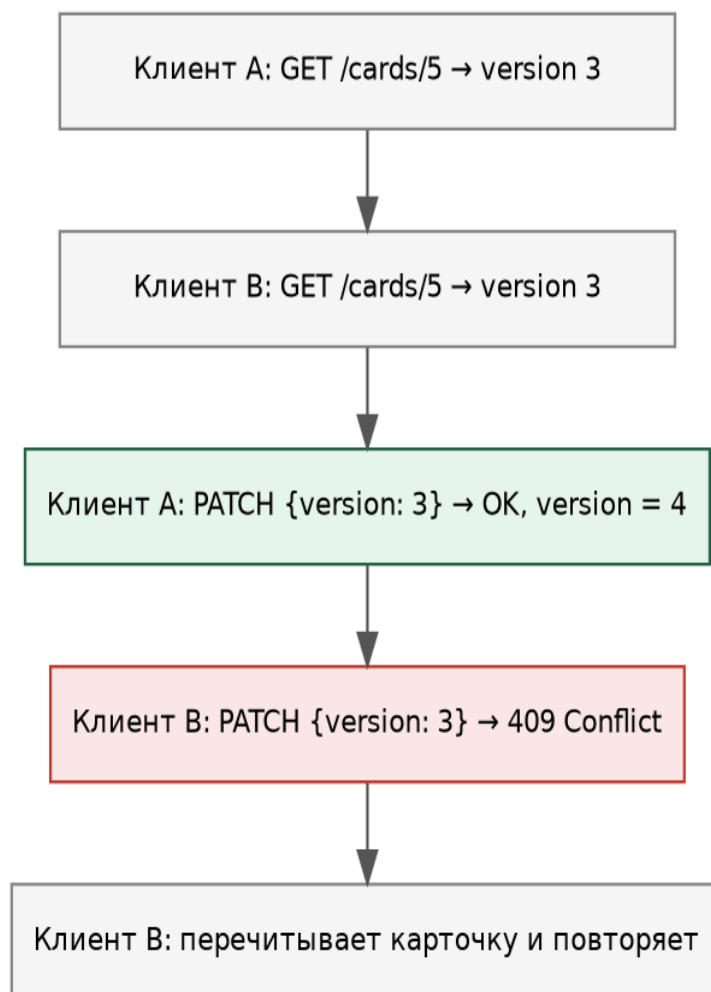


Рис. 4. Защита от коллизий (оптимистичная блокировка)

Данный подход, в отличие от пессимистичной блокировки, не удерживает записи базы данных заблокированными на время работы клиента и лучше масштабируется при большом числе пользователей, что оправдано при относительно редких конфликтах на доске. Реализация перемещения карточки приведена в Листинге 5.

```
def move_card(db, card, user, board_id,
              target_column_id, position, version):
    check_version(card, version) # 409 при несовпадении
    target = db.get(Column, target_column_id)
    if target is None or target.board_id != board_id:
        raise HTTPException(400, 'Колонка не с этой доски')
    card.column_id = target_column_id
    card.position = position
    card.version += 1
    audit_service.log(db, board_id=board_id, user_id=user.id,
                      entity='card', entity_id=card.id,
                      action='move')
    db.commit() # единая транзакция
    return card
```

Листинг 5. Перемещение карточки в одной транзакции

Порядок карточек в колонке

Позиция карточки внутри колонки хранится как число с плавающей точкой. Такой подход, известный как дробное индексирование, позволяет вставлять карточку между двумя соседними, вычисляя её позицию как среднее арифметическое позиций соседей. Благодаря этому перемещение карточки изменяет ровно одну запись и не требует пересчёта позиций остальных карточек колонки. Принцип показан на Рис. 5.

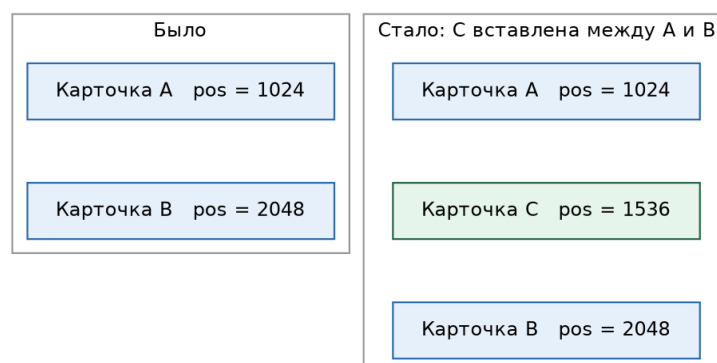


Рис. 5. Дробное индексирование позиций карточек

3.5 Журнал изменений

Все изменения доски — создание, редактирование, перемещение и удаление сущностей, а также изменение состава участников — фиксируются в таблице журнала. Каждая запись содержит идентификатор пользователя, тип сущности, выполненное действие и полезную нагрузку с описанием изменения в формате JSONB. Запись в журнал выполняется в той же транзакции, что и основная операция, поэтому журнал всегда согласован с данными. Просмотр журнала доступен только владельцу доски.

3.6 Клиентское приложение

Клиентская часть реализована в виде одностраничного приложения на библиотеке React. Приложение включает страницу входа и регистрации, список досок пользователя и рабочую область доски. Внешний вид рабочей области приведён на Рис. 6.

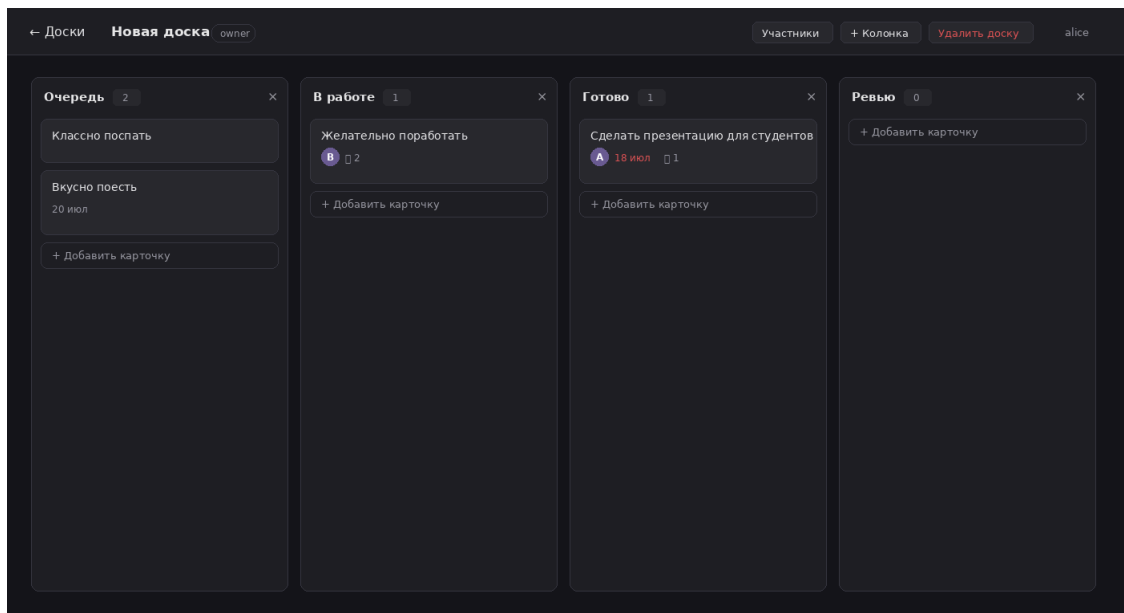


Рис. 6. Рабочая область доски

Перемещение карточек между колонками и внутри колонки реализовано методом перетаскивания с использованием библиотеки `@dnd-kit`. Применяется оптимистичное обновление интерфейса: карточка визуально перемещается немедленно, не дожидаясь ответа сервера, а при получении

ответа о конфликте состояние доски перечитывается. Это обеспечивает отзывчивость интерфейса без потери согласованности данных.

Детали карточки — описание, исполнитель, срок выполнения и комментарии — открываются в модальном окне. Пользователю с ролью читателя элементы редактирования не отображаются, что дополняет серверную проверку прав на уровне интерфейса.

3.7 Развёртывание

Развёртывание системы выполняется средствами Docker Compose и включает три контейнера: базу данных, серверную часть и клиентское приложение с веб-сервером nginx. Схема развёртывания приведена на Рис. 7.

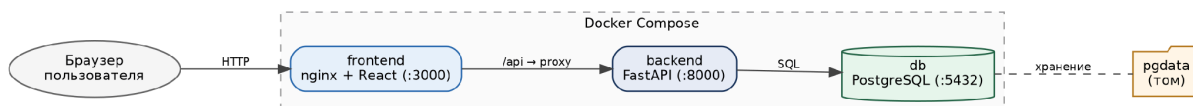


Рис. 7. Схема развёртывания системы

При первом запуске серверный контейнер автоматически применяет миграции базы данных и выполняет скрипт наполнения демонстрационными данными. Данные базы сохраняются в отдельном томе, что обеспечивает их сохранность при перезапуске контейнеров. Запуск всей системы выполняется одной командой, приведённой в Листинге 6.

```
docker compose up --build

# приложение: http://localhost:3000
# документация API: http://localhost:8000/docs
```

Листинг 6. Команда запуска системы

4 Тестирование

Проверка работоспособности системы выполнялась по сценариям, охватывающим как основную функциональность, так и механизмы разграничения доступа и защиты от коллизий. Тестирование проводилось через программный интерфейс с использованием демонстрационных учётных записей с различными ролями. Результаты приведены в Таблице 7.

Таблица 7 — Результаты проверки сценариев

Проверяемый сценарий	Ожидаемый результат	Итог
Регистрация нового пользователя	Учётная запись создана	Пройдено
Вход по логину и паролю	Выдан токен доступа	Пройдено
Создание доски и колонок	Объекты созданы	Пройдено
Создание карточки	Присвоены позиция и версия	Пройдено
Перемещение карточки	Смена колонки, рост версии	Пройдено
Перемещение с устаревшей версией	Конфликт (409)	Пройдено
Добавление комментария	Комментарий сохранён	Пройдено
Запись пользователем-читателем	Отказ в доступе (403)	Пройдено
Обращение к чужой доске	Не найдено (404)	Пройдено
Запрос без токена	Требуется вход (401)	Пройдено
Фиксация действий в журнале	Записи созданы	Пройдено

Отдельное внимание было уделено проверке сценария конкурентного редактирования. Два клиента получали одну и ту же карточку с одинаковой версией, после чего последовательно отправляли изменения. Первый запрос выполнялся успешно с увеличением версии, второй отклонялся с кодом 409, что подтверждает корректность работы механизма оптимистичной блокировки.

Также проверялась изоляция данных между пользователями: учётная запись, не являющаяся участником доски, не получала доступ ни к самой доске, ни к её колонкам, карточкам и комментариям.

Заключение

В ходе практики спроектирован и реализован полнофункциональный веб-сервис управления задачами на основе Kanban-досок. Выполнены все требования обязательного минимума: реализованы регистрация и авторизация пользователей, управление досками, колонками и карточками, перемещение карточек методом перетаскивания с сохранением порядка, назначение исполнителей и сроков выполнения, комментирование задач и ролевое разграничение доступа. Дополнительно реализован журнал изменений, относящийся к возможным расширениям решения.

В процессе работы были отработаны следующие практические навыки: проектирование реляционной модели данных с корректными связями, ключами и индексами; построение программного интерфейса в стиле REST с аутентификацией на основе токенов и ролевой моделью разграничения доступа; обеспечение транзакционной целостности данных и защиты от конкурентных изменений средствами оптимистичной блокировки; разработка клиентского приложения с интерактивным интерфейсом; подготовка воспроизводимого развёртывания многосервисного приложения.

Принятые архитектурные решения обоснованы характером предметной области. Выбор модели RBAC вместо ABAC продиктован отсутствием динамических атрибутивных правил доступа. Применение оптимистичной блокировки вместо пессимистичной обусловлено редкостью конфликтов и требованием масштабируемости. Использование дробного индексирования позиций позволило свести перемещение карточки к изменению одной записи.

В качестве направлений дальнейшего развития можно выделить: добавление меток и чек-листов карточек, поддержку вложений, реализацию полнотекстового поиска по карточкам, введение шаблонов досок, а также переход на обновление интерфейса в реальном времени с использованием веб-сокетов, что позволит участникам видеть изменения других пользователей без перезагрузки страницы.

За время прохождения практики теоретические знания, полученные в университете, были применены при решении практической задачи разработки программного обеспечения.

Список использованных источников

- 1) Документация фреймворка FastAPI, <https://fastapi.tiangolo.com/>
- 2) Документация СУБД PostgreSQL, <https://www.postgresql.org/docs/>
- 3) PostgreSQL: Transactions, <https://www.postgresql.org/docs/current/tutorial-transactions.html>
- 4) Документация библиотеки SQLAlchemy, <https://docs.sqlalchemy.org/>
- 5) Документация Alembic, <https://alembic.sqlalchemy.org/>
- 6) Introduction to JSON Web Tokens, <https://www.jwt.io/introduction>
- 7) Документация библиотеки React, <https://react.dev/learn>
- 8) Документация библиотеки @dnd-kit, <https://docs.dndkit.com/>
- 9) Документация Docker Compose, <https://docs.docker.com/compose/>
- 10) Сравнение моделей управления доступом RBAC и ABAC, <https://habr.com/ru/companies/custis/articles/248649/>